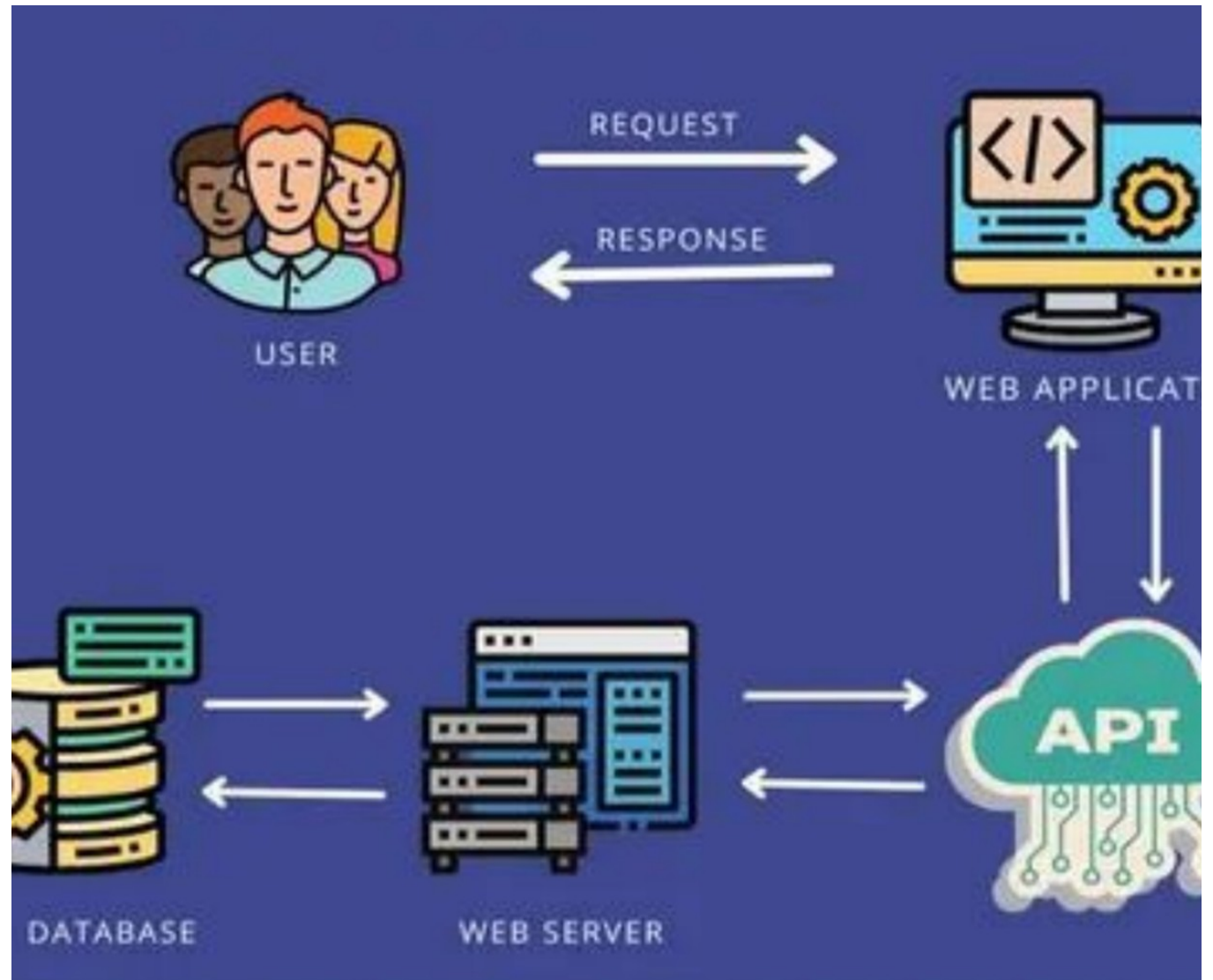


API

Application Programming Interface



API - Application Programming Interface

- Application Programming Interfaces (APIs) are constructs made available in programming languages to allow developers to create complex functionality more easily.
- allows communication between software components.
- It defines rules and protocols for accessing a web-based software application.
- They abstract more complex code away from you, providing some easier syntax to use in its place.

Why Do We Need APIs?

- Reuse functionality (no need to build everything from scratch).
- Connect frontend ↔ backend.
- Share data between apps (e.g., your app uses Google Maps API).
- Standardize communication using **HTTP protocol**.

How API Works (Basic Flow)

- **Client** (browser/app) → sends **HTTP Request**
Example: GET /users/1
- **Server** (API provider) → processes request
- **Response** → API sends back data (usually JSON)

Four main categories of API

- Open (Public)
- Partner (Restricted)
- Internal (Private)
- Composite

1. Open APIs (Public APIs)

- Accessible by anyone (usually with registration or API key).
- Examples:
 - OpenWeather API
 - GitHub REST API
 - JSONPlaceholder (free testing API)
- **Use case:** Public data sharing, integrations.

2. Partner APIs (Restricted APIs)

- Shared with specific business partners.
- Requires **special permission / license / contracts**.
- Example:
 - Travel booking company gives API access only to partner hotels.
 - Payment gateway APIs given only to registered merchants.
- **Use case:** Business collaborations.

3. Internal APIs (Private APIs)

- Used **inside a company** only.
- Not exposed to outsiders.
- Example:
 - Company's HR system talking with Payroll system.
 - Internal microservices communicating in a big product.
- **Use case:** Increase efficiency, reuse functionality across internal teams.

4. Composite APIs

- Combines **multiple APIs into one request**.
- Saves time → instead of calling 3 APIs separately, you call **1 composite API** that fetches from all.
- Example:
 - A travel app API that combines flight + hotel + cab booking APIs.
- **Use case:** Complex workflows, microservices.

API Testing

- It is a type of software testing that validates APIs directly to ensure they return correct responses for various requests and inputs

Protocols are used in *API* testing?

- HTTP
- HTTPS
- SOAP
- REST
- GraphQL
- WebSocket
- JMS

APIs in client-side JavaScript

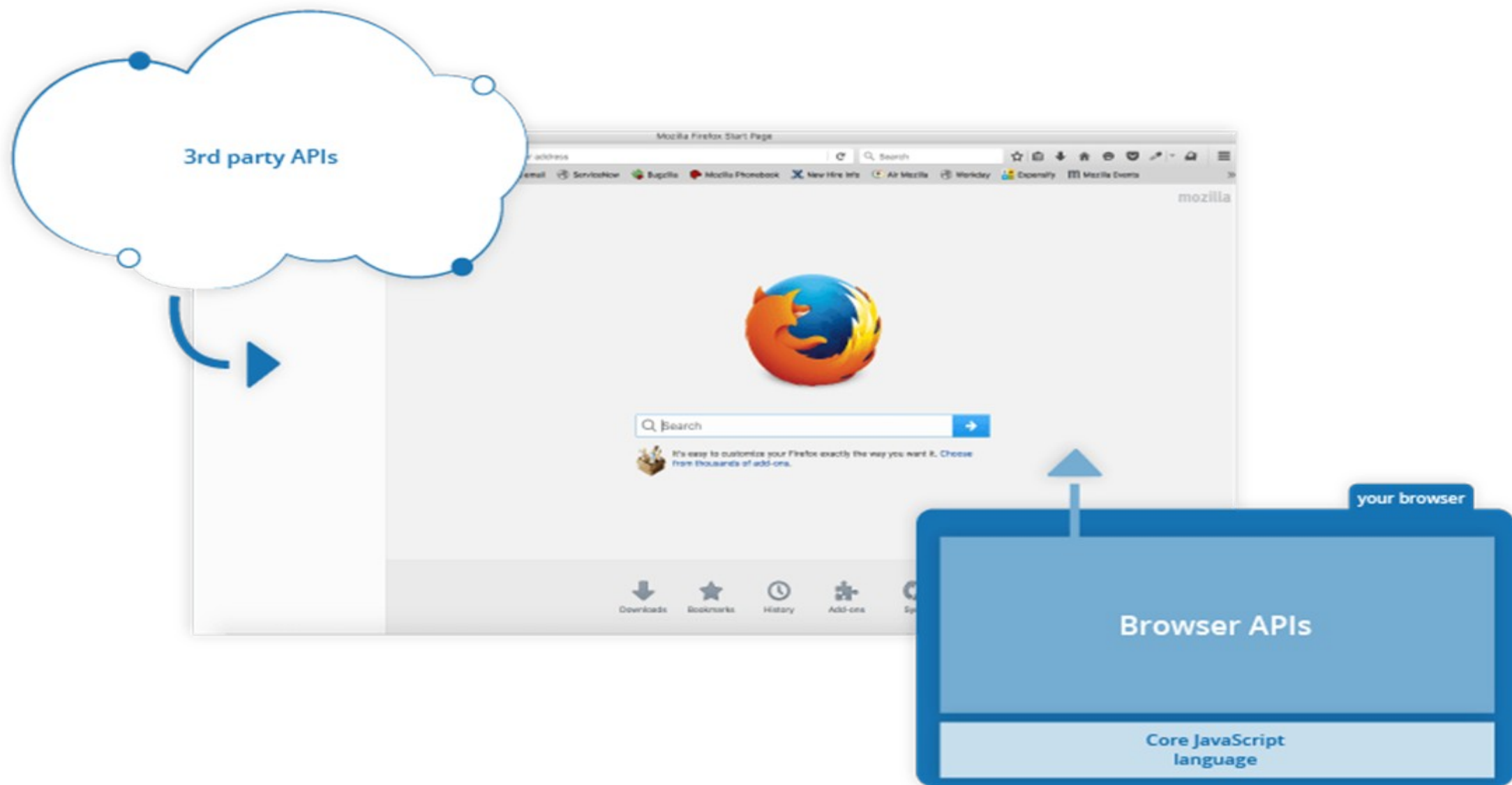
- Client-side JavaScript, in particular, has many APIs available to it — these are not part of the JavaScript language itself, rather they are built on top of the core JavaScript language, providing you with extra superpowers to use in your JavaScript code.
- They generally fall into two categories:
 - **Browser APIs**
 - **Third-party APIs**

Browser APIs

- It are built into your web browser and are able to expose data from the browser and surrounding computer environment and do useful complex things with it.
1. APIs for manipulating documents loaded into the browser.
 2. APIs that fetch data from the server to update small sections of a webpage
 3. APIs for drawing and manipulating graphics are widely supported in browsers
 4. Audio and Video APIs
 5. Device APIs enable you to interact with device hardware
 6. Client-side storage APIs enable you to store data on the client-side

Third-party APIs

- Third-party APIs are not built into the browser by default, and you generally have to retrieve their code and information from somewhere on the Web.
- For example, the Twitter API allows you to do things like displaying your latest tweets on your website.
- It provides a special set of constructs you can use to query the Twitter service and return specific information



Third-party APIs

- **Twitter API** - which allows you to do things like displaying your latest tweets on your website.
- **Map APIs** - which allow you to do all sorts of things with maps on your web pages.
- **Telegram APIs** - which allows you to embed content from Telegram channels on your website, in addition to providing support for bots.
- **YouTube API** - which allows you to embed YouTube videos on your site, search YouTube, build playlists, and more.

Benefits Of Using API

- APIs needed to make the connection between application functions to handle the data from predefined processes
- **Easy of Integration:** APIs can be embedded with any type of software application, so it can be easily integrate with the applications and functions between different websites

Working with APIs in JavaScript

- An API is simply a medium to fetch or send data between interfaces.
- Some real-time data fetched from the server or maybe even allows you to modify or add data to some other endpoint. This is made possible by the API or the Application Programming Interface

REST vs RESTful APIs

What is REST?

- **REST = Representational State Transfer**
- It's **not a technology**, but an **architectural style / set of guidelines** for designing APIs.
- Proposed by **Roy Fielding** in his PhD dissertation (2000).
- Defines **how clients and servers should communicate** over HTTP.
- **Think of REST as the “theory / rules” behind designing APIs.**

REST vs RESTful APIs

- **2. RESTful API**
- A **RESTful API** is an **actual API implementation** that **follows REST rules**.
- Not every API using HTTP is RESTful.
- To be RESTful, an API must follow **all the constraints of REST**.
- **Think of RESTful API as the “practical application” of REST rules.**

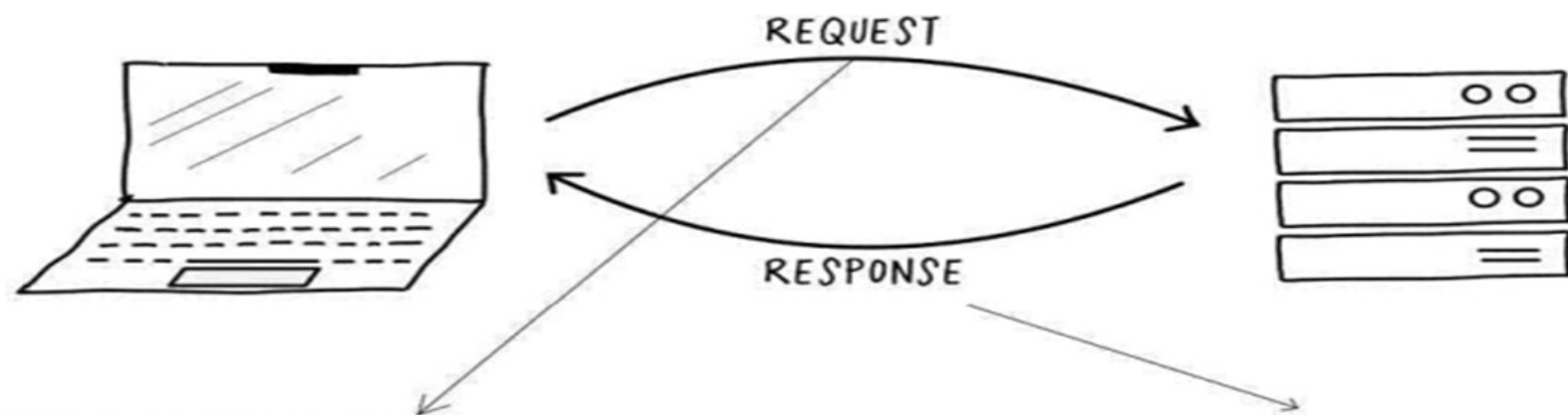
REST vs RESTful APIs

Feature	REST	RESTful API
Definition	Architectural principles / rules	API that follows REST rules
Nature	Theoretical concept	Practical implementation
Focus	Guidelines for designing APIs	Actual API design
Example	“Use resources, statelessness, HTTP methods”	/users/1 (GET, POST, DELETE, etc.)

REST API

REST STANDS FOR REPRESENTATIONAL STATE TRANSFER

REST APIS OPERATE ON A SIMPLE REQUEST/RESPONSE SYSTEM

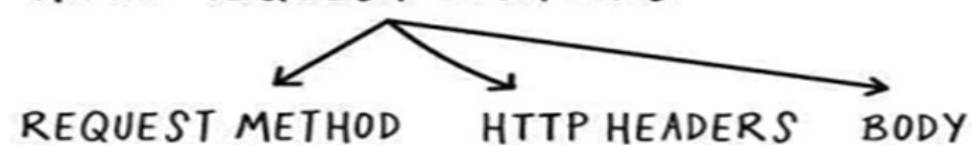


CLIENT CAN MAKE A REQUEST
USING HTTP METHODS

THESE METHODS ARE:

GET, POST, PUT, PATCH, DELETE, HEAD,
TRACE, OPTIONS, CONNECT

HTTP REQUEST CONTAINS

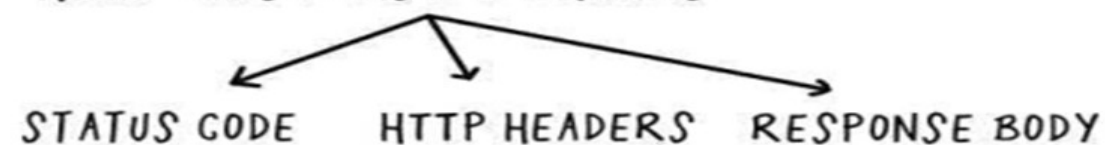


SERVER RETURNS A RESPONSE WITH AN
HTTP STATUS CODE

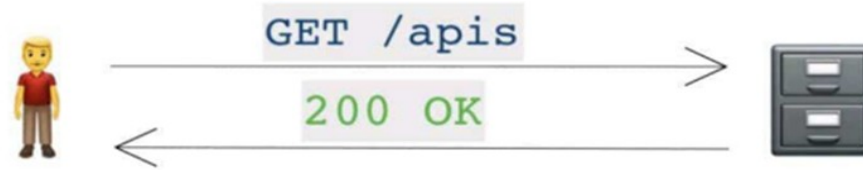
POPULAR HTTP STATUS CODE:

EX, 200, 202, 403, 404, 500 ETC

HTTP RESPONSE CONTAINS



HTTP Request Methods



GET

The GET method is the most common of all these request methods. It is used to fetch the desired resources from the server.

The POST method is used to submit the information to the server. As we're submitting data, the POST method often changes the state of the server.

POST

PUT

The PUT method is used whenever you need to change the resource. The resource, which is already a part of resource collection.

The PATCH request method is used to modify only the necessary part of the data or response.

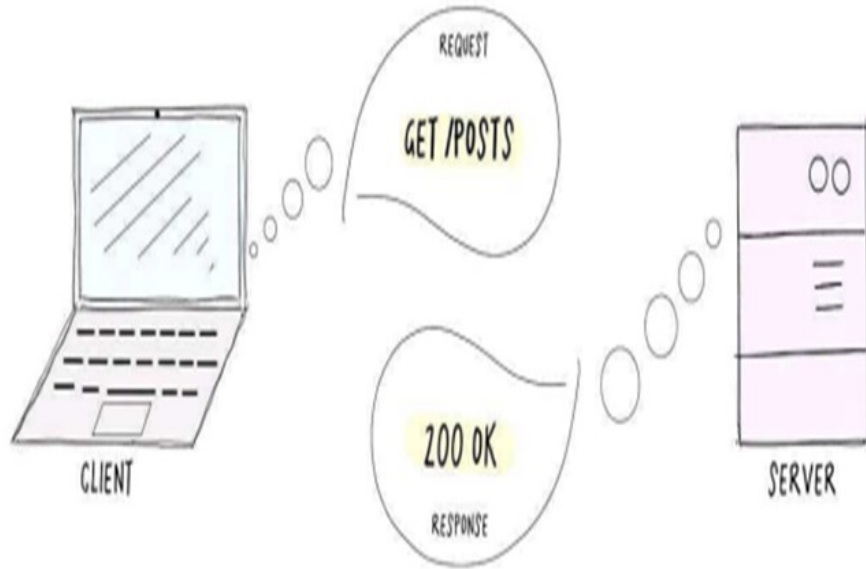
The PATCH method doesn't modify the entire response.

PATCH

DELETE

As the name says, the DELETE request method is used to delete the specified resource. It requests that the origin server delete the resource identified by the Request-URL.

HTTP STATUS CODES



★ SERVER ALWAYS RETURNS HTTP STATUS CODE WITH THE RESPONSE ★

SUCCESSFUL RESPONSES

200 OK

EVERYTHING IS FINE

201 CREATED

NEW RESOURCE WAS CREATED

REDIRECTION MESSAGES

301 MOVED PERMANENTLY

THE RESOURCE HAS BEEN MOVED PERMANENTLY
TO THE NEW URL

CLIENT ERROR

400 BAD REQUEST
INVALID SYNTAX

401 UNAUTHORIZED
CREDENTIALS ARE INCORRECT

403 FORBIDDEN
YOU DON'T HAVE PERMISSION TO ACCESS THE RESOURCES

404 NOT FOUND
INVALID URL

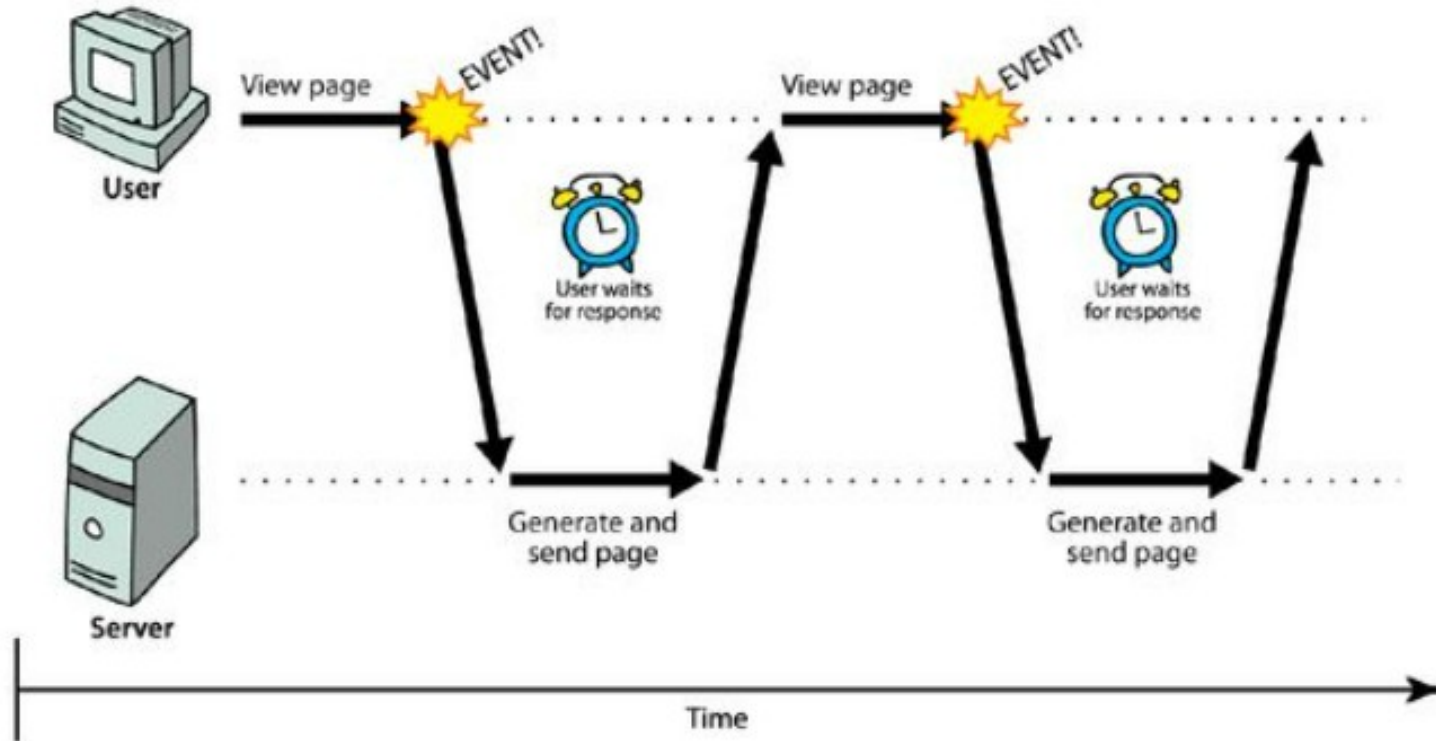
429 TOO MANY REQUESTS
USER HAS SENT TOO MANY REQUESTS IN A
GIVEN AMOUNT OF TIME

SERVER ERROR

500 INTERNAL SERVER ERROR

SERVER DOES NOT KNOW HOW TO HANDLE THE UNEXPECTED SITUATION

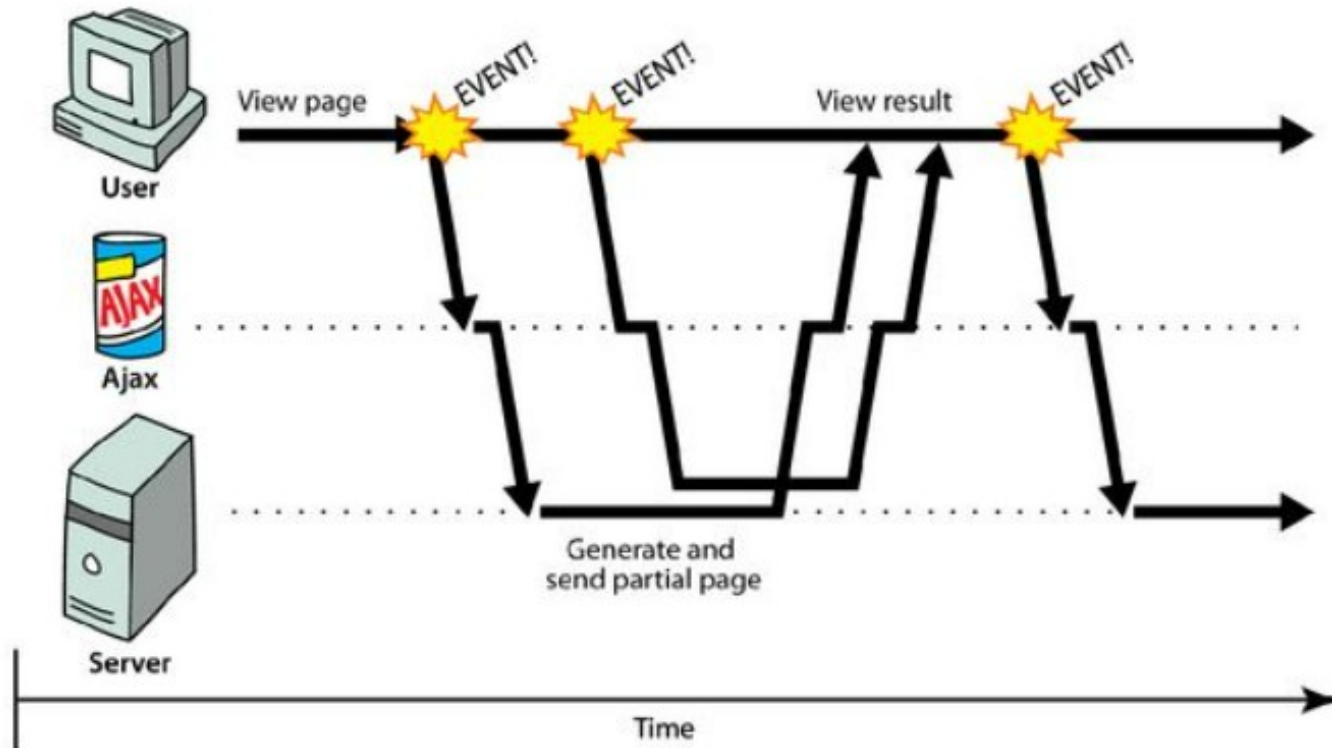
Synchronous Web Communication



Synchronous Web Communication

- User must wait while new pages load
- The typical communication pattern used in web pages (click, wait, refresh)
- As the name suggests synchronous means to be in a sequence, i.e. every statement of the code gets executed one by one. So, basically a statement has to wait for the earlier statement to get executed.
- This can lead to blocking behavior, where the execution of subsequent code is delayed until the current task is finished.
- Eg: `console.log("Task 1");`
 `console.log("Task 2");`
 `console.log("Task 3");`

Asynchronous Web Communication



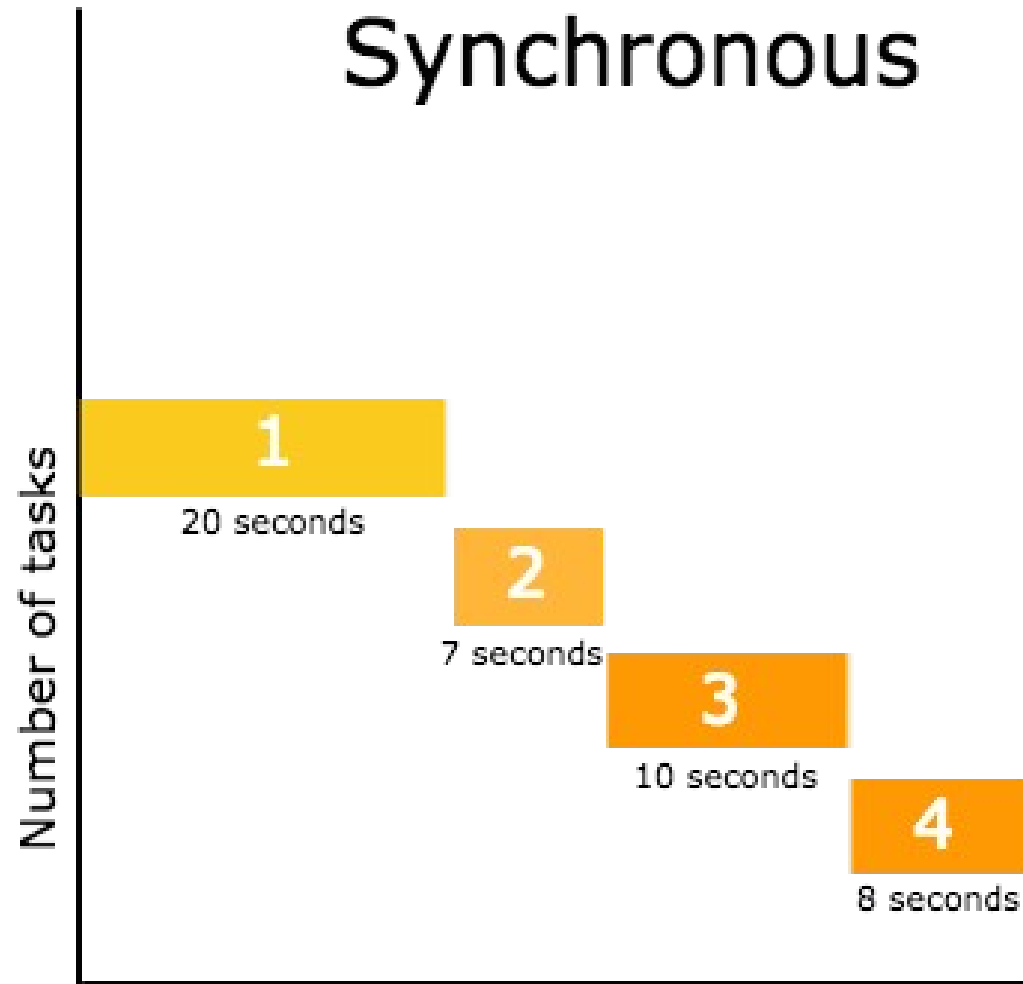
Asynchronous Web Communication

- In asynchronous JavaScript, tasks are executed independently of the main program flow.
- When an asynchronous task is encountered, it is initiated and the program continues to execute without waiting for it to complete.
- Callback functions or promises are typically used to handle the result of asynchronous tasks when they finish executing.
- Asynchronous JavaScript is particularly useful for tasks that might take some time to complete, such as fetching data from a server, reading files, or waiting for user input. It allows the program to continue executing other tasks without being blocked by long-running operations.

- Eg:

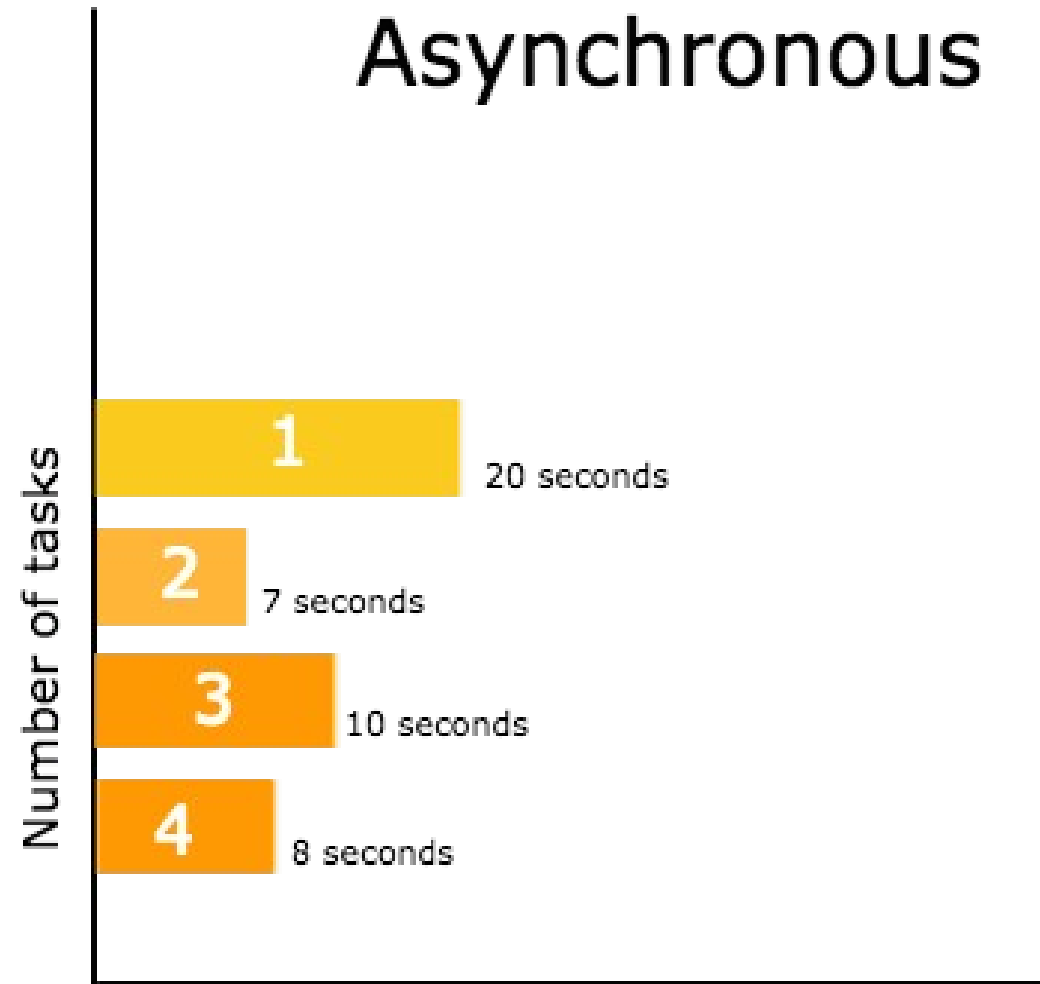
```
console.log("Task 1");  
setTimeout(function() {  
    console.log("Task 2");  
}, 1000);  
console.log("Task 3");
```

Synchronous



Total time taken by the tasks.
45 seconds

Asynchronous



Total time taken by the tasks.
20 seconds

Callback Functions

- A callback function is a function that is passed as an argument to another function, to be “called back” at a later time.
- A function that accepts other functions as arguments is called a higher-order function, which contains the logic for when the callback function gets executed.
- It’s the combination of these two that allow us to extend our functionality.
- Eg:

```
setTimeout(function() {  
    console.log("This function executes after the specified delay.");  
}, 1000);
```

AJAX

Asynchronous Javascript And Xml

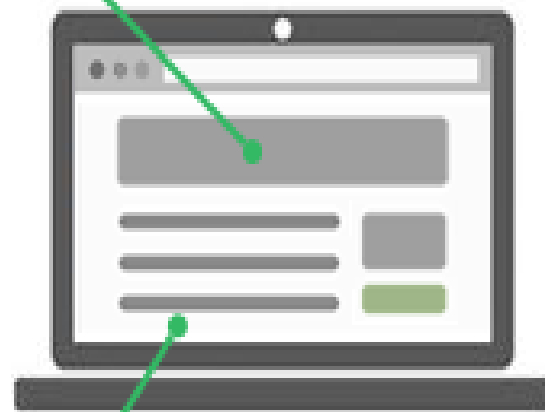
AJAX

- AJAX is not a programming language.
- AJAX just uses a combination of: A browser built-in XMLHttpRequest object (to request data from a web server) JavaScript and HTML DOM (to display or use the data)
- Ajax is just a means of loading data from the server and selectively updating parts of a web page without reloading the whole page.
- Basically, what Ajax does is make use of the browser's built-in XMLHttpRequest (XHR) object to send and receive information to and from a web server asynchronously, in the background, without blocking the page or interfering with the user's experience.
- The example of some large-scale Ajax-driven online applications are: Gmail, Google Maps, Google Docs, YouTube, Facebook, Flickr, and so many other applications.

An event is occurred on a web page
e.g. a button is clicked

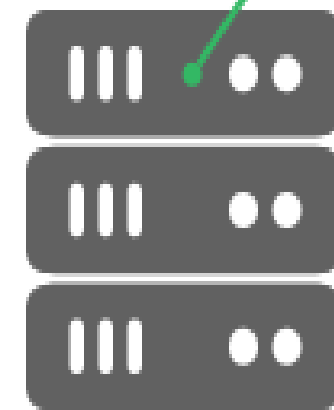
An HTTP request is sent to server
using **XMLHttpRequest** object

Process the HTTP request
and creates a response



Browser

Network



Server

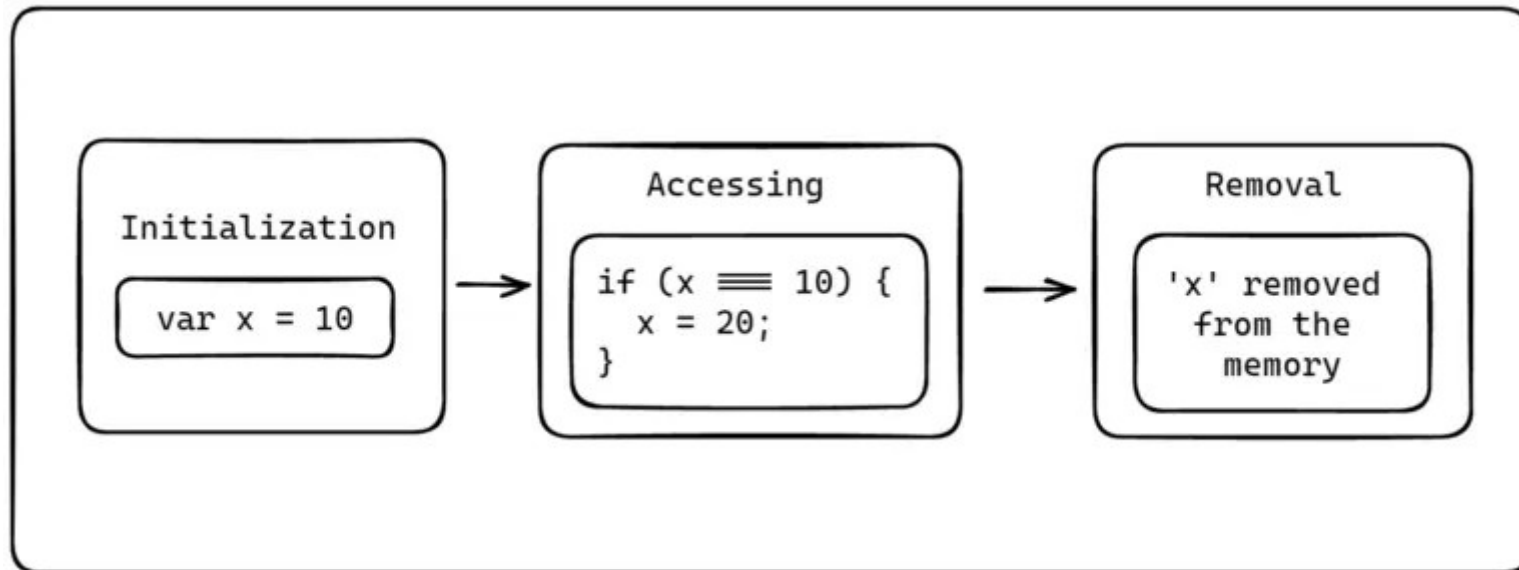
Process the returned data using JavaScript
and updates the page content

Send response back
to the browser

Memory Management in JavaScript

Memory life cycle in JavaScript

- The memory life cycle defines a JavaScript variable or object's journey from initialization to removal. JavaScript and all the other programming languages have a similar memory life cycle, as depicted below.

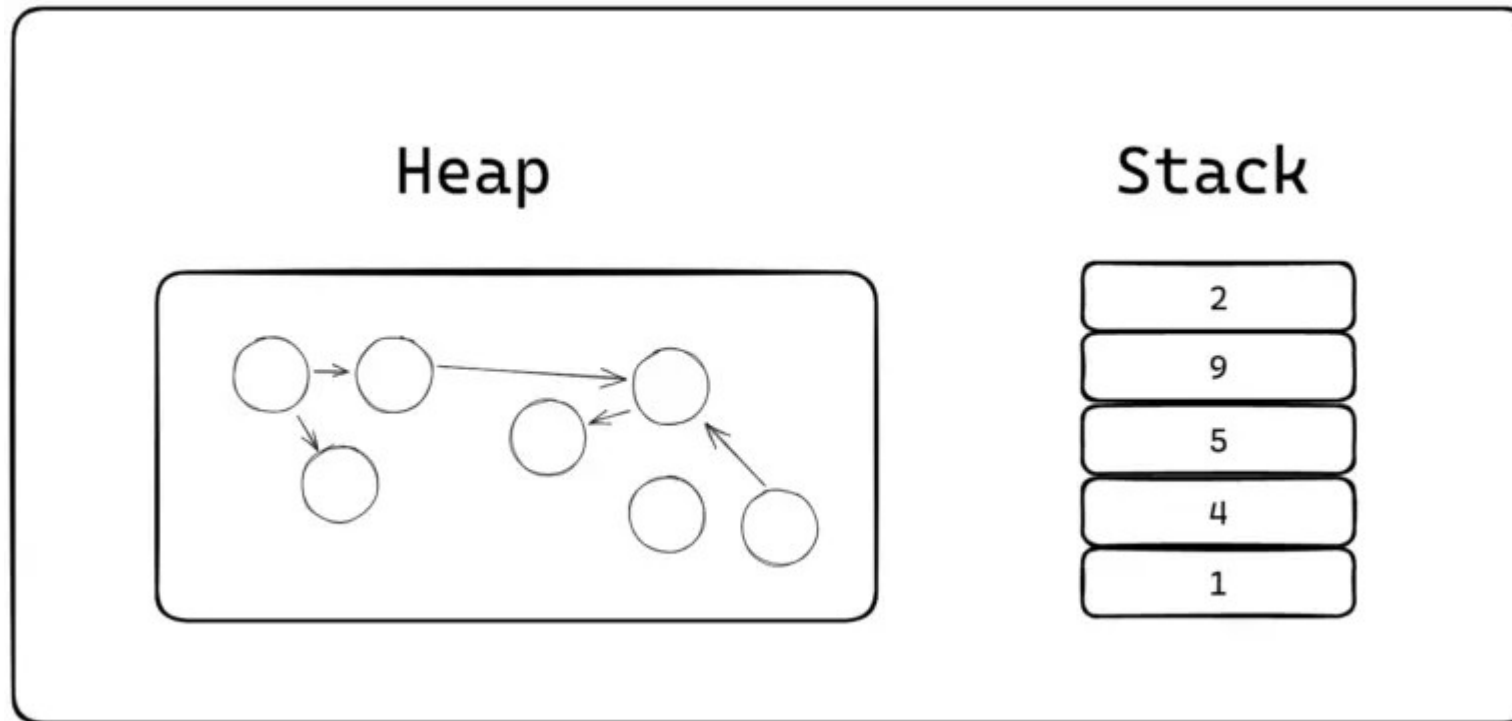


How does JavaScript store variables?

Where do these allocations and deallocation happen?

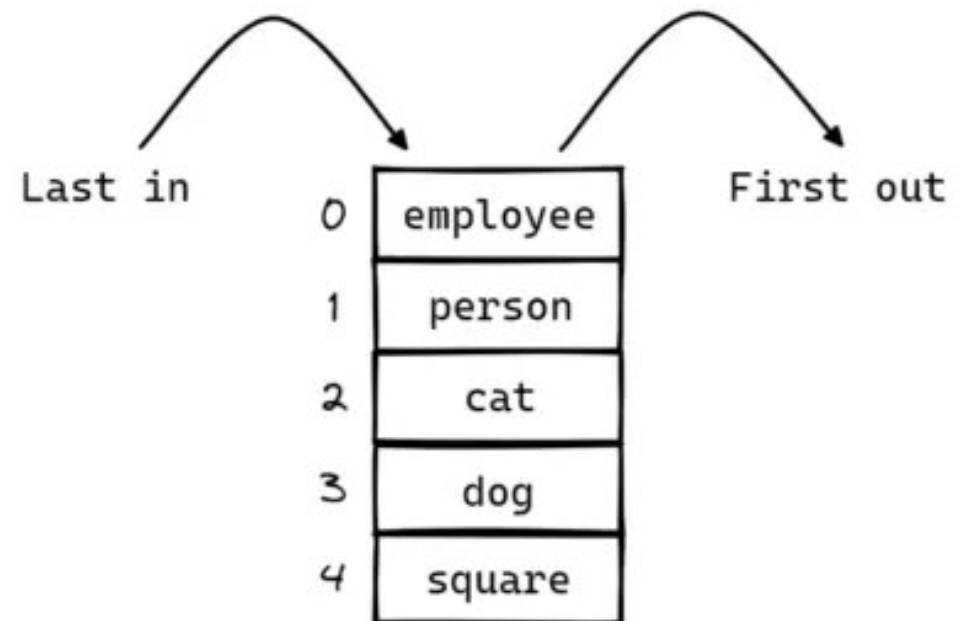
How does JavaScript store variables?

Where do these allocations and deallocation happen?



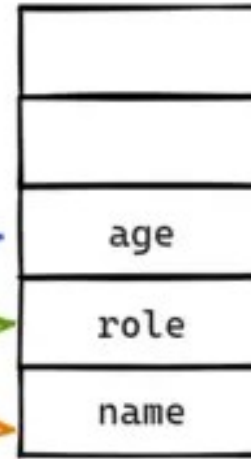
Stack

- JavaScript uses the stack data structure to store static or fixed-size data.
- It includes all numbers, strings, booleans, and other primitive data types.
- These data types have a fixed size known at compile time.
- Variables such as objects, arrays, etc., are not stored in the stack as their size varies during run time.



```
1 let name = 'Varun';  
2 let role = 'Writer';  
3 const age = 21;
```

Stack



Order of
deallocation



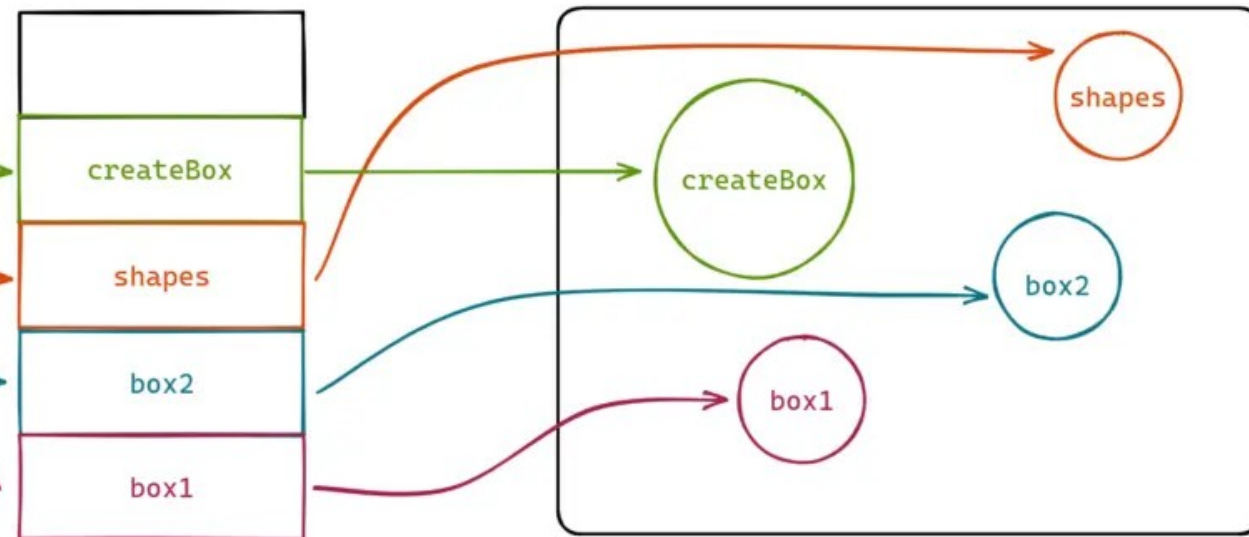
Heap

- JavaScript uses a heap for storing variables whose size is unknown at compile time or may vary at the run time, such as objects, arrays, functions, etc.
- The JavaScript Engine dynamically allocates memory to the heap. Initially, the heap size depends on available system memory, and it can be dynamically increased/decreased based on the need.

Values and references

- JavaScript allocates memory for objects within the heap, but we must have a reference to that memory location to access the value.
- The reference to the memory location resides in the stack memory.
-

```
1 let box1 = {  
2   width: 100,  
3   height: 50  
4 };  
5  
6 let box2 = {  
7   width: 120,  
8   height: 80,  
9 }  
10  
11 const shapes = ['square', 'circle', 'triangle', 'rectangle'];  
12  
13 function createBox (width, height) {  
14   return {  
15     width: width,  
16     height: height  
17   }  
18 }
```



Stack Overflow

- When the function runs inside and inside, the call stack will be filled and overflows.
- When the stack overflows, Maximum call stack size exceeded error will be thrown.
- The below function runs inside and inside and the stack will be overflowed.

Ex: function inception() {

 inception();

}

inception();

Garbage Collection

- JavaScript automatically cleans out the memory allocated to the data after its execution.
- For example when the memory allocated for the variable created inside the function, it will be cleared after the function execution automatically.
- It will manages the memory leaks.
- It will automatically controls the memory heap.
- It follows Mark and Sweep algorithm to handle Garbage collection.

Memory Leaks

- Memory leaks occurred when the data is overloaded (Infinite data into an array), it breaks the browser.

Ex: let array = [];

```
for(let i=1; i>0; i++) {
```

```
    array.push(i);
```

```
}
```

Call Stack

- The Call Stack is a part of the JavaScript Engine and it is simply a stack in which you can add an item and the item added first is processed last.
- In other words, it follows the FILO - 'First In Last Out' principle.
- A Call Stack acts as a placeholder or a holding area for all the JavaScript functions that have been fired for execution.

What is Event Loop in JavaScript?

- The event loop is a fundamental concept in JavaScript for managing asynchronous operations.
- It's a mechanism that allows JavaScript to handle tasks such as timers, I/O operations, and event handling in a non-blocking manner, ensuring smooth and responsive behavior in web applications.
- following components that constitute Event Loop Architecture.
 - 1 The Call Stack
 - 2 Web API
 - 3 Event Queue

Call Stack

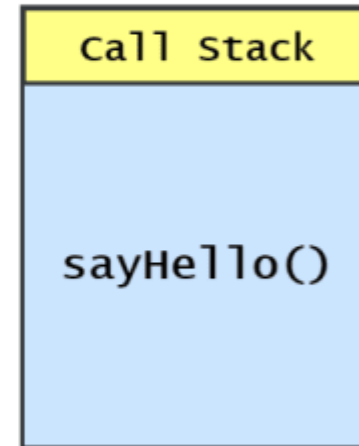
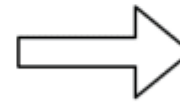
- Let us look at the below code sample and understand how the function 'sayHello' gets added to the Call Stack.

```
const sayHello = () => {  
  console.log('Hello from Skay');  
}  
  
sayHello();
```

Step 1 - sayHello Function gets added to the Call Stack

1 - Functions are pushed to the callstack when they are invoked.

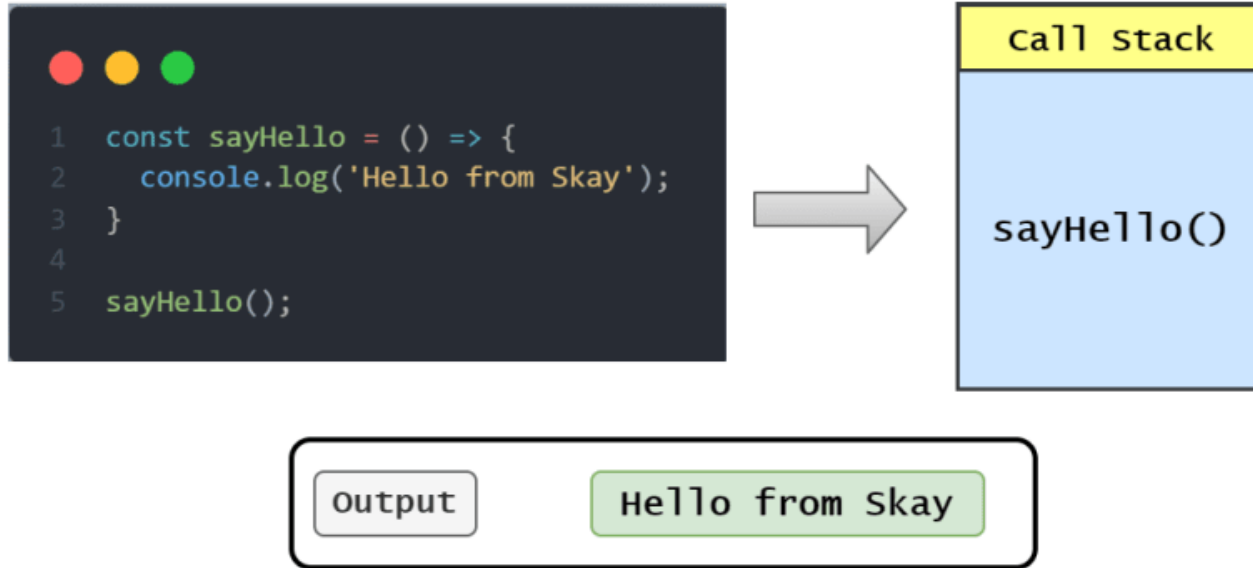
```
1  const sayHello = () => {  
2    console.log('Hello from Skay');  
3  }  
4  
5  sayHello();
```



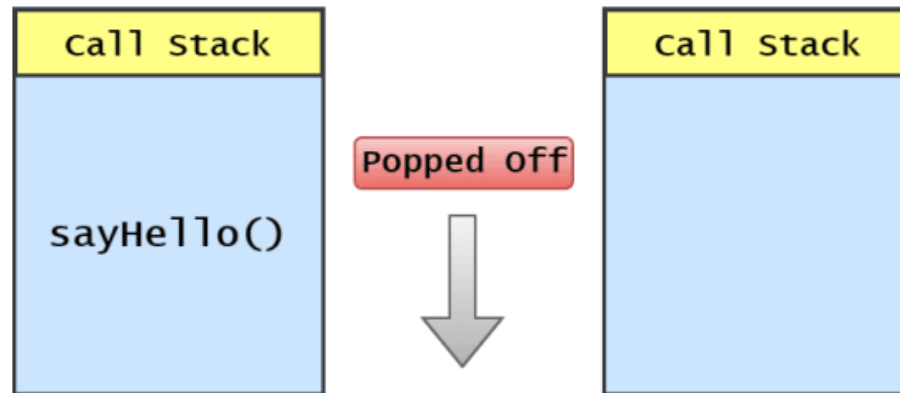
Step 2 & 3 - Function returns a value & gets popped from the Call Stack

- Once the function returns a value, the output 'Hello from Skay' gets displayed on the console. As soon as the function returns, almost immediately, the function is also popped off from the call stack.

2 - Function returns value as output on the console.



3 - Functions are popped off the callstack after they return a value.



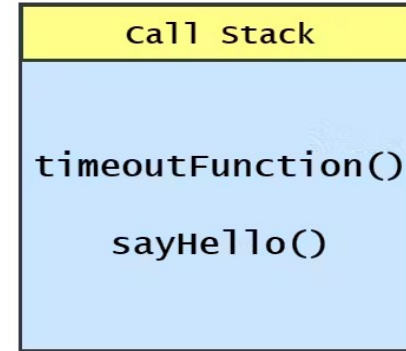
```
const sayHello = () => {  
  console.log('Hello from Skay');  
}
```

```
const timeoutFunction = () => {  
  return setTimeout(() => {  
    console.log('Hello from Timeout Function');  
  }, 1000)  
}
```

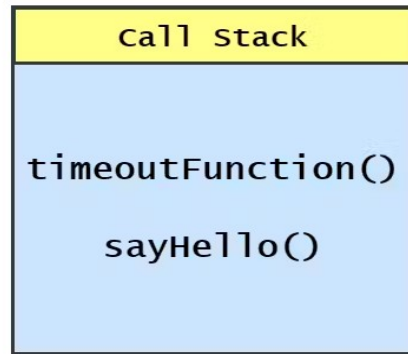
```
sayHello();  
timeoutFunction();
```

1 - sayHello is added first to the stack, followed by timeoutFunction

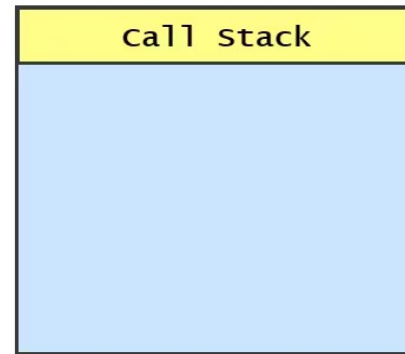
```
1  const sayHello = () => {  
2    console.log('Hello from Skay');  
3  }  
4  
5  const timeoutFunction = () => {  
6    return setTimeout(() => {  
7      console.log('Hello from Timeout Function');  
8    }, 1000)  
9  }  
10  
11 sayHello();  
12 timeoutFunction();
```



2 - sayHello & timeFunction are popped off the call stack after they complete execution/return a value.



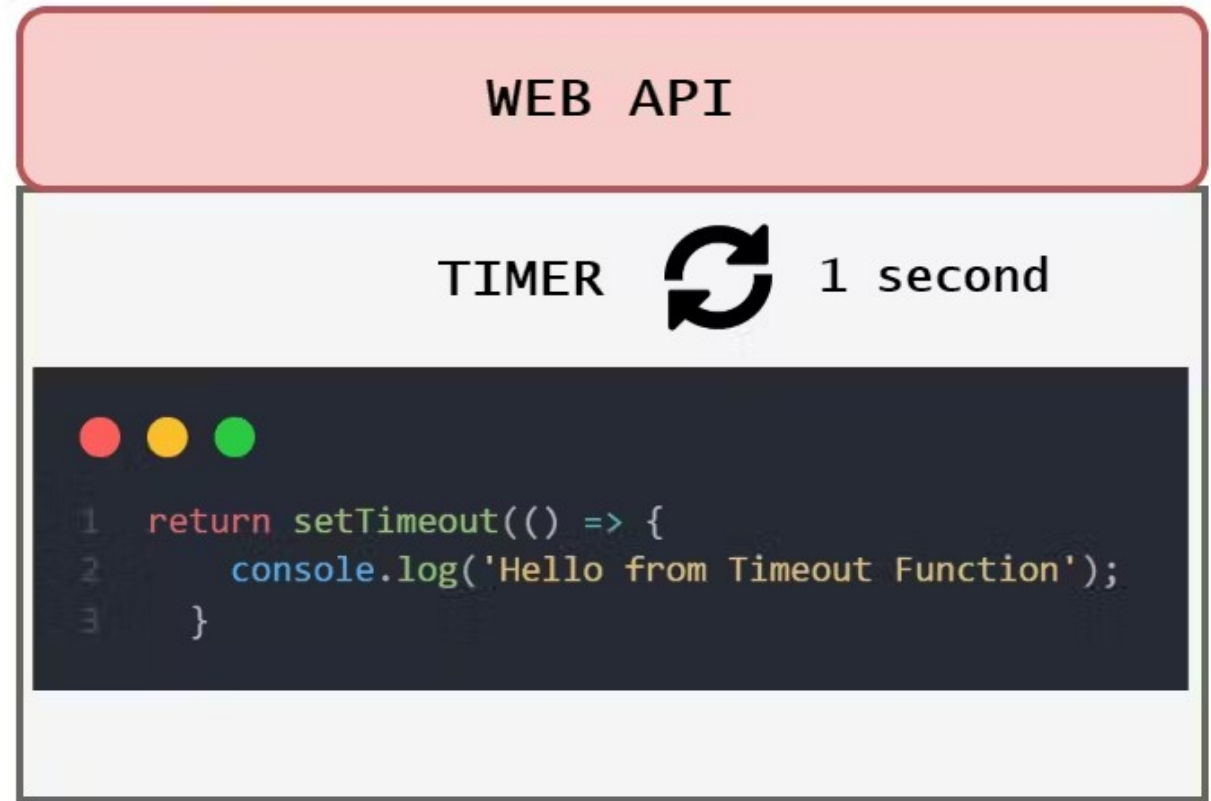
Popped off



Web API

- Web APIs are built into your web browser and they are able to process data from the browser to do complex useful things. The main thing to understand is that, they are not a part of the JavaScript language, but, they are built on top of the core JS language and provide much more capabilities such as Geolocation, localStorage, etc.

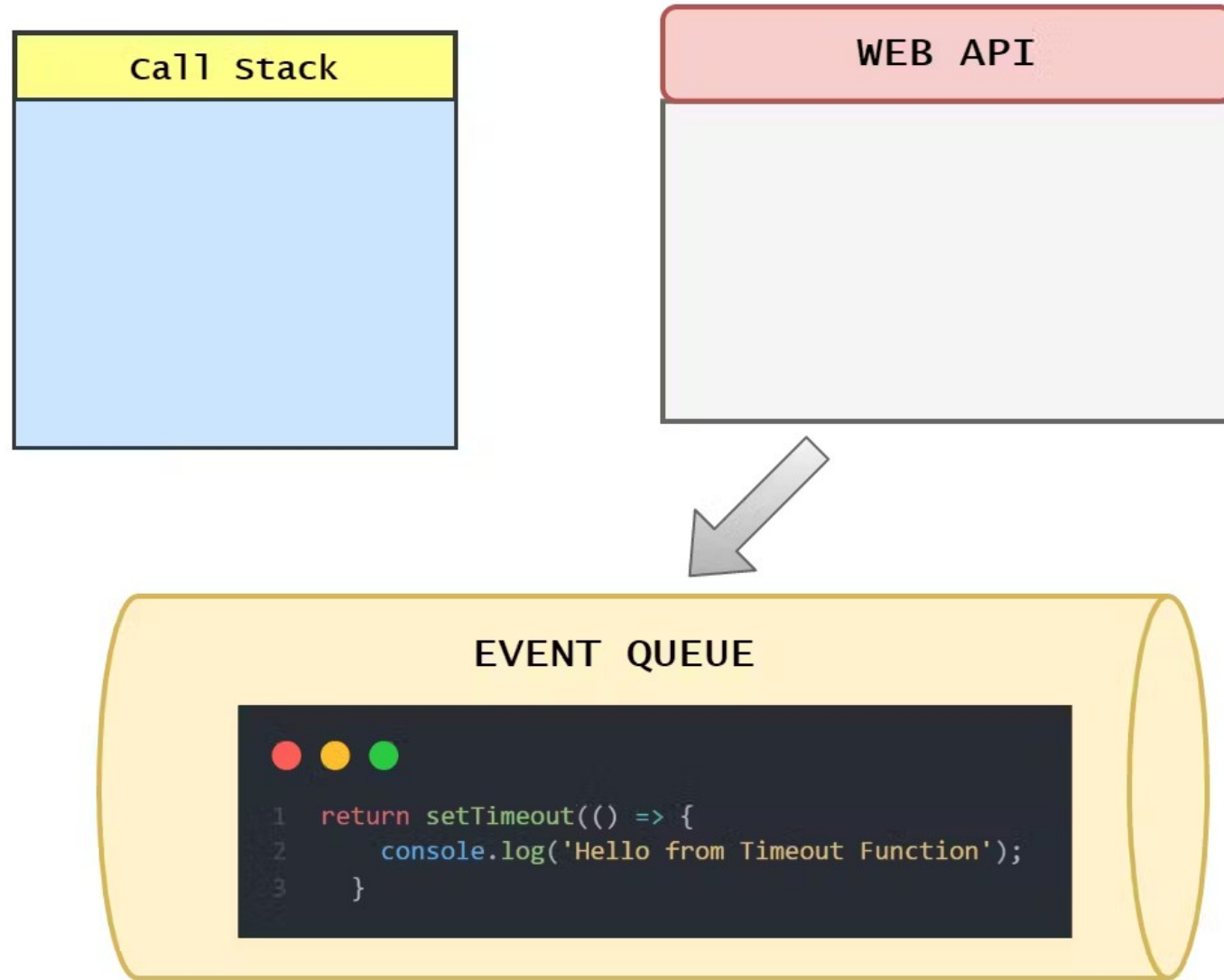
3 - The setTimeout function is added to WebAPI with the timer set to 1 second



Event Queue

- Event Queue is a data structure similar to Stack, which holds the data temporarily and the important thing to note is that the data added first is processed first. In other words, it follows the FIFO -> 'First in First out' principle.

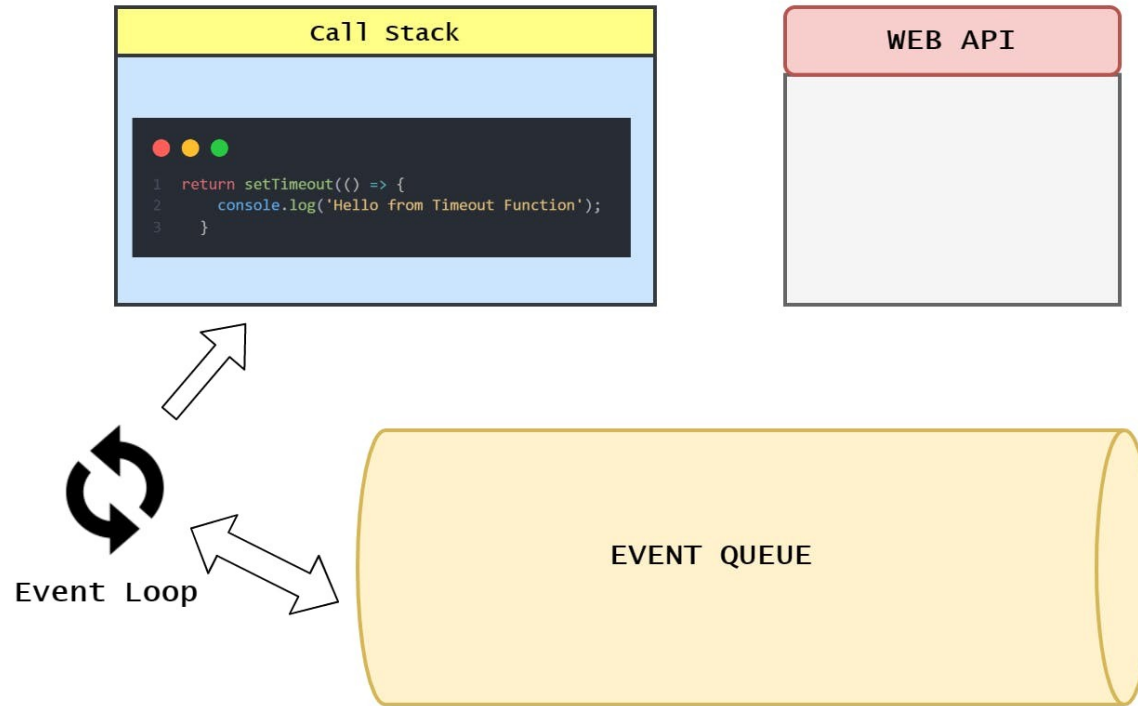
4 - After 1 second, the `setTimeout` function is added to the Event Queue



Event Loop

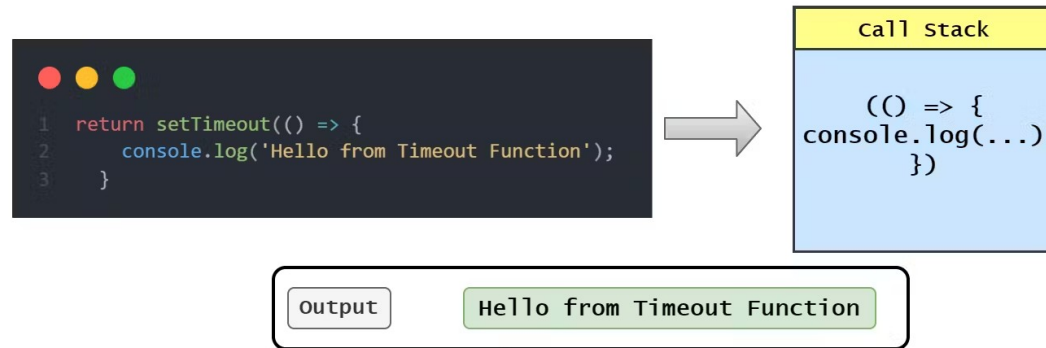
- Finally, the 'Event Loop' we've been waiting for. The function of Event Loop can simply be explained as connecting the Event Queue to the Call Stack.

5 - The Event loop moves the setTimeout Function from the Event Queue to the Call Stack



- Event Loop does the following:
- Checks if the Call Stack is empty, i.e., if all the functions have completed their execution and they have been popped off the Call Stack.
- Once the Call Stack is empty, it moves the first item from the Event Queue to the Call Stack.

6 - Anonymous Function returns value as output on the console.



7 - Anonymous Function popped off the callstack after execution.

